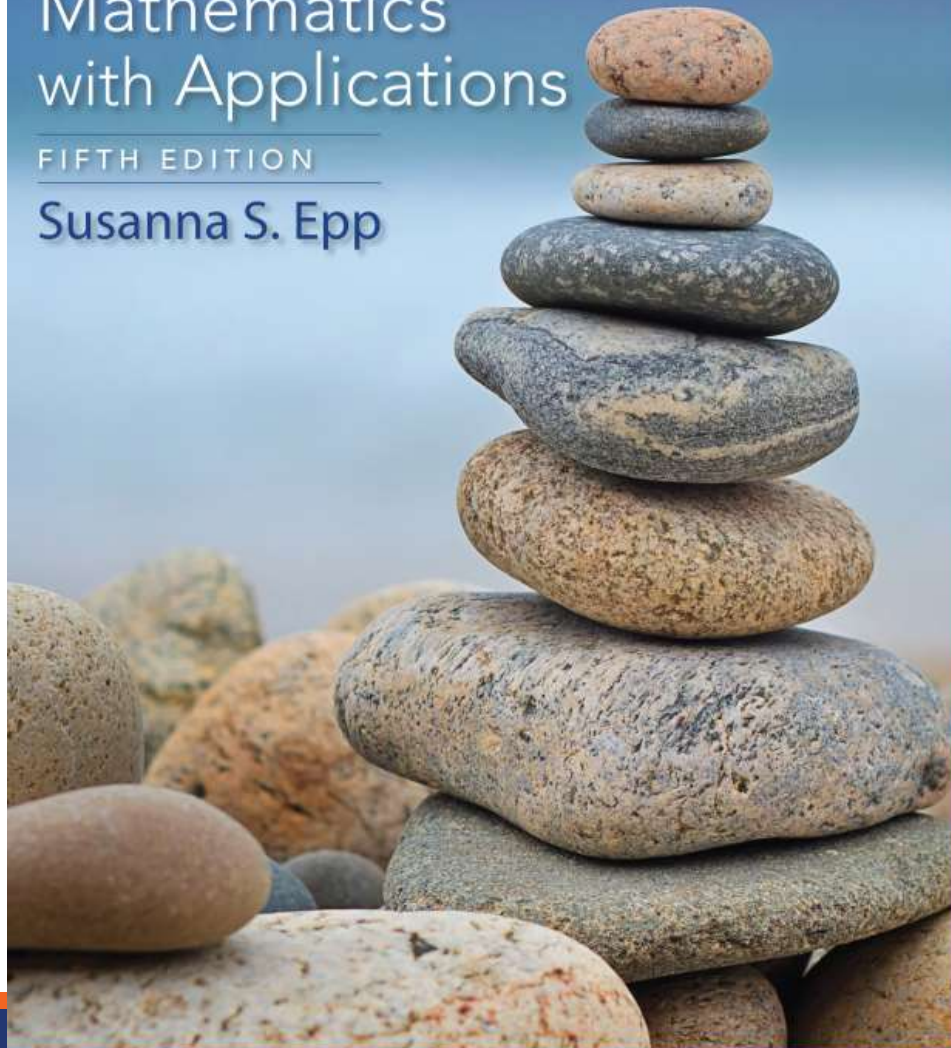


Discrete Mathematics with Applications

FIFTH EDITION

Susanna S. Epp





Faculty of Computer Science

UFCFFS-30-1– Foundations of Computing: Analysis of Algorithms & Algorithm Complexity & Big-O notations

□ Instructor: Duc Minh Vu

Learning Objectives

1. Real-Valued Functions of a Real Variable and Their Graphs
2. Big-O, Big-Omega, and Big-Theta Notations
3. Analysis of Algorithm Efficiency I
4. Exponential and Logarithmic Functions: Graphs and Orders
5. Analysis of Algorithm Efficiency II

Reading

- ❑ Chapter 11 - Discrete Mathematics with Applications 5th Edition
(Susanna's Book)
- ❑ Chapter 3 – Discrete Mathematics and Its Applications 7th Edition
(Kenneth Rosen's book)



Application: Analysis of Algorithm Efficiency I

11.3 Application: Analysis of Algorithm Efficiency I

□ *The Sequential Search Algorithm*

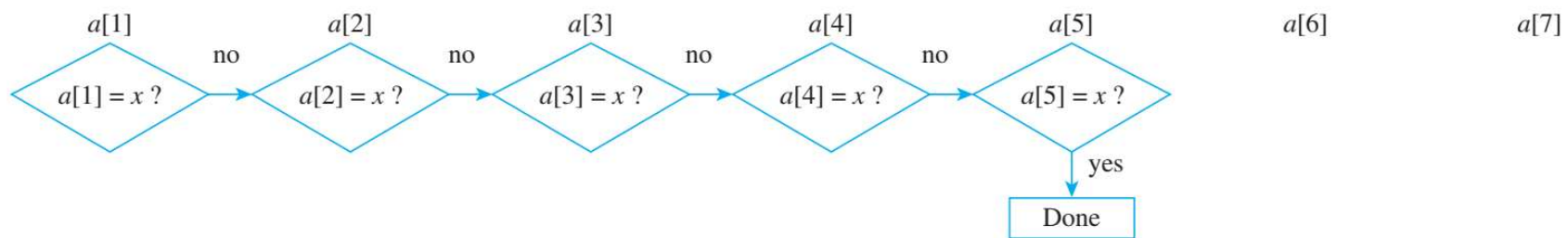


FIGURE 11.3.1 Sequential Search of $a[1], a[2], \dots, a[7]$ for x where $x = a[5]$

□ *Best-, worst-, and Average-Case Orders for Sequential Search*

➤ $O(1)$, $O(n)$ and $O(n)$

Measuring the Efficiency of an Algorithm

- ❑ Two aspects of algorithm efficiency are important: time required to execute the algorithm and memory space needed when it is run.
- ❑ The analysis of an algorithm for time efficiency count the number of elementary operations that must be performed when the algorithm is executed with an input of size n .
- ❑ Elementary operations: addition, subtraction, multiplication, division, and comparisons

TABLE 11.3.1 Number of Elementary Operations for Algorithms *A* and *B*

	<i>Max A</i>	<i>Min B</i>	<i>Max B – Max A</i>
<i>n</i>	$20n$	$2n^2$	$2n^2 - 20n$
1,000	20,000	2,000,000	1,980,000
10,000	200,000	200,000,000	199,800,000
1,000,000	20,000,000	2,000,000,000,000	1,999,980,000,000

Definition

Let A be an algorithm.

1. Suppose the number of elementary operations performed when A is executed for an input of size n depends on n alone and not on the nature of the input data; say it equals $f(n)$. If $f(n)$ is $\Theta(g(n))$, we say that **A is $\Theta(g(n))$ or A is of order $g(n)$.**
2. Suppose the number of elementary operations performed when A is executed for an input of size n depends on the nature of the input data as well as on n .
 - a. Let $b(n)$ be the *minimum* number of elementary operations required to execute A for all possible input sets of size n . If $b(n)$ is $\Theta(g(n))$, we say that **in the best case, A is $\Theta(g(n))$ or A has a best-case order of $g(n)$.**
 - b. Let $w(n)$ be the *maximum* number of elementary operations required to execute A for all possible input sets of size n . If $w(n)$ is $\Theta(g(n))$, we say that **in the worst case, A is $\Theta(g(n))$ or A has a worst-case order of $g(n)$.**

TABLE 11.3.2 Time Comparisons of Some Algorithm Orders

Approximate Time to Execute $f(n)$ Operations Assuming One Operation per Nanosecond*				
$f(n)$	$n = 10$	$n = 1,000$	$n = 100,000$	$n = 10,000,000$
$\log_2 n$	3.3×10^{-9} sec	10^{-8} sec	1.7×10^{-8} sec	2.3×10^{-8} sec
n	10^{-8} sec	10^{-6} sec	0.0001 sec	0.01 sec
$n \log_2 n$	3.3×10^{-8} sec	10^{-5} sec	0.0017 sec	0.23 sec
n^2	10^{-7} sec	0.001 sec	10 sec	27.8 hr
n^3	10^{-6} sec	1 sec	11.6 days	31,688 yr
2^n	10^{-6} sec	3.4×10^{284} yr	3.1×10^{30086} yr	2.9×10^{3010283} yr

*one nanosecond = 10^{-9} second, one year $\cong$ 365.25 days

Example 11.3.2

Computing an Order of an Algorithm Segment

Assume n is a positive integer and consider the following algorithm segment:

$p := 0, x := 2$

for $i := 2$ **to** n

$p := (p + i) \cdot x$

next i

- Compute the actual number of elementary operations that are performed when this algorithm segment is executed.
- Use the theorem on polynomial orders to find an order for this algorithm segment.

Solution

- a. This algorithm segment has just one loop, which goes from 2 to n , and so there are as many iterations of the loop as there are integers from 2 to n , namely $n - 2 + 1 = n - 1$.^{*} During each iteration, one multiplication and one addition are performed. Thus twice as many elementary operations are performed as there are iterations of the loop, and hence $2(n - 1) = 2n - 2$ elementary operations are performed when the algorithm segment is executed.
- b. By the theorem on polynomial orders,

$$2n - 2 \text{ is } \Theta(n),$$

and so this algorithm segment is $\Theta(n)$. ■

Example 11.3.3

An Order for an Algorithm with a Nested Loop

Assume n is a positive integer and consider the following algorithm segment:

```

 $s := 0$ 
for  $i := 1$  to  $n$ 
    for  $j := 1$  to  $i$ 
        if  $\left\lfloor \frac{i}{j} \right\rfloor \cdot j = i$  then  $s := s + 1$ 
    next  $j$ 
next  $i$ 

```

- Compute the actual number of elementary operations that are performed when this algorithm segment is executed.
- Use the theorem on polynomial orders to find an order for this algorithm segment.

Example 11.3.4

When the Number of Iterations Depends on the Floor Function

Assume n is a positive integer and consider the following algorithm segment:

```
for  $i := \lfloor n/2 \rfloor$  to  $n$ 
     $a := n - i$ 
next  $i$ 
```

- Compute the actual number of elementary operations that are performed when this algorithm segment is executed.
- Use the theorem on polynomial orders to find an order for this algorithm segment.

TEST YOURSELF

1. When an algorithm segment contains a nested **for-next** loop, you can find the number of times the loop will iterate by constructing a table in which each column represents _____.
2. In the worst case for an input array of length n , the sequential search algorithm has to look through _____ elements of the array before it terminates.
3. The worst-case order of the insertion sort algorithm is _____, and its average-case order is _____.

EXERCISE SET 11.3

1. Suppose a computer takes 1 nanosecond ($=10^{-9}$ second) to execute each operation. Approximately how long will it take the computer to execute the following numbers of operations? Convert your answers into seconds, minutes, hours, days, weeks, or years, as appropriate. For example, instead of 2^{50} nanoseconds, write 13 days.
 - a. $\log_2 200$
 - b. 200
 - c. $200 \log_2 200$
 - d. 200^2
 - e. 200^8
 - f. 2^{200}
2. Suppose an algorithm requires cn^2 operations when performed with an input of size n (where c is a constant).
 - a. How many operations will be required when the input size is increased from m to $2m$ (where m is a positive integer)?
 - b. By what factor will the number of operations increase when the input size is doubled?
 - c. By what factor will the number of operations increase when the input size is increased by a factor of ten?

3. Suppose an algorithm requires cn^3 operations when performed with an input of size n (where c is a constant).
- a. How many operations will be required when the input size is increased from m to $2m$ (where m is a positive integer)?
 - b. By what factor will the number of operations increase when the input size is doubled?
 - c. By what factor will the number of operations increase when the input size is increased by a factor of ten?

Exercises 4–5 explore the fact that for relatively small values of n , algorithms with larger orders can be more efficient than algorithms with smaller orders.

4. Suppose that when run with an input of size n , algorithm A requires $2n^2$ operations and algorithm B requires $80n^{3/2}$ operations.
 - a. What are orders for algorithms A and B from among the set of power functions?
 - b. For what values of n is algorithm A more efficient than algorithm B ?
 - c. For what values of n is algorithm B at least 100 times more efficient than algorithm A ?
5. Suppose that when run with an input of size n , algorithm A requires $10^6 n^2$ operations and algorithm B requires n^3 operations.
 - a. What are orders for algorithms A and B from among the set of power functions?
 - b. For what values of n is algorithm A more efficient than algorithm B ?
 - c. For what values of n is algorithm B at least 100 times more efficient than algorithm A ?

For each of the algorithm segments in 6–19, assume that n is a positive integer. (a) Compute the actual number of elementary operations (additions, subtractions, multiplications, divisions, and comparisons) that are performed when the algorithm segment is executed. For simplicity, however, count only comparisons that occur within **if-then** statements; ignore those implied by **for-next** loops. (b) Use the theorem on polynomial orders to find an order for the algorithm segment.

6. **for** $i := 3$ **to** $n - 1$
 $a := 3 \cdot n + 2 \cdot i - 1$
next i
7. $max := a[1]$
 for $i := 2$ **to** n
 if $max < a[i]$ **then** $max := a[i]$
 next i

8. $a := 0$
for $i := 1$ **to** $\lfloor n/2 \rfloor$
 $a := a + 3$
next i
9. $s := 0$
for $i := 1$ **to** n
for $j := 1$ **to** $2n$
 $s := s + i \cdot j$
next j
next i
10. **for** $k := 2$ **to** n
for $j := 1$ **to** $3n$
 $x := a[k] - b[j]$
next j
next k
11. **for** $k := 1$ **to** $n - 1$
for $j := 1$ **to** $k + 1$
 $x := a[k] + b[j]$
next j
next k
12. **for** $k := 1$ **to** $n - 1$
 $max := a[k]$
for $i := k + 1$ **to** n
if $max < a[i]$ **then** $max := a[i]$
next i
 $a[k] := max$
next k

13. for $i := 1$ to $n - 1$

for $j := i$ to n

if $a[j] > a[i]$ then do

$temp := a[i]$

$a[i] := a[j]$

$a[j] := temp$

end do

next j

next i

15. for $i := 1$ to $n - 1$

$p := 1$

$q := 1$

for $j := i + 1$ to n

$p := p \cdot c[j]$

$q := q \cdot (c[j])^2$

next j

$r := p + q$

next i

14. $t := 0$

for $i := 1$ to n

$s := 0$

for $j := 1$ to i

$s := s + a[j]$

next j

$t := t + s^2$

next i

16. for $i := 1$ to n

$s := 0$

for $j := 1$ to $i - 1$

$s := s + j \cdot (i - j + 1)$

next j

$r := s^2$

next i

H* 19. for $i := 1$ to n

for $j := 1$ to i

for $k := 1$ to j

$x := i \cdot j \cdot k$

next k

next j

next i

17. for $i := 1$ to n

for $j := 1$ to $\lfloor (i+1)/2 \rfloor$

$a := (n - i) \cdot (n - j)$

next j

next i

20. Construct a table showing the result of each step when insertion sort is applied to the array $a[1] = 6$, $a[2] = 2$, $a[3] = 1$, $a[4] = 8$, and $a[5] = 4$.

18. for $i := 1$ to n

for $j := \lfloor (i + 1)/2 \rfloor$ to n

$x := i \cdot j$

next j

next i

21. Construct a table showing the result of each step when insertion sort is applied to the array $a[1] = 7$, $a[2] = 3$, $a[3] = 6$, $a[4] = 9$, and $a[5] = 5$.



Exponential and Logarithmic Functions: Graphs and Orders

x	2^x
0	$2^0 = 1$
1	$2^1 = 2$
2	$2^2 = 4$
3	$2^3 = 8$
-1	$2^{-1} = 0.5$
-2	$2^{-2} = 0.25$
-3	$2^{-3} = 0.125$
0.5	$2^{0.5} \cong 1.414$
-0.5	$2^{-0.5} \cong 0.707$

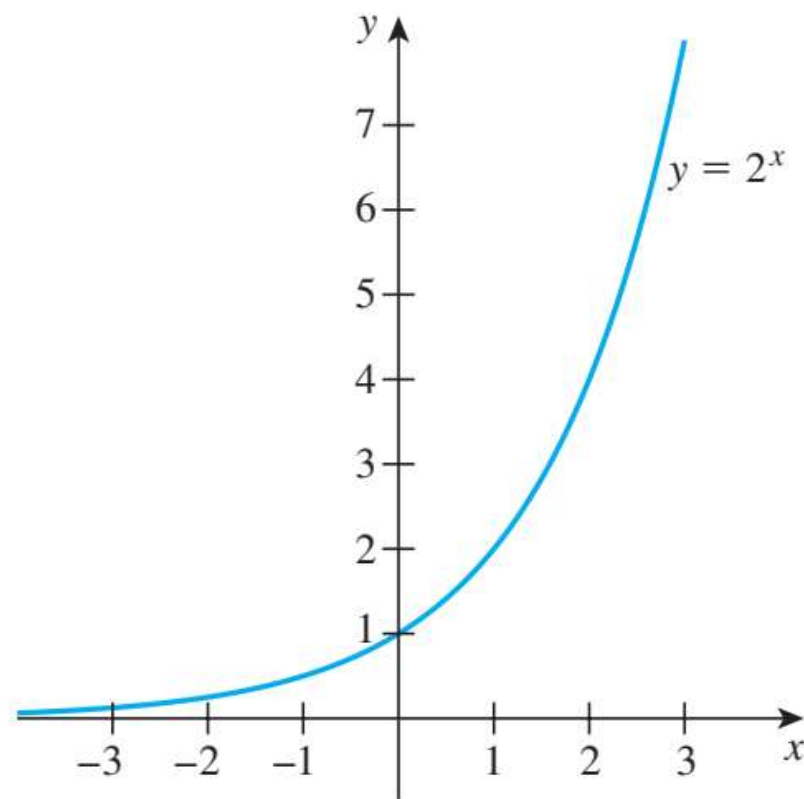
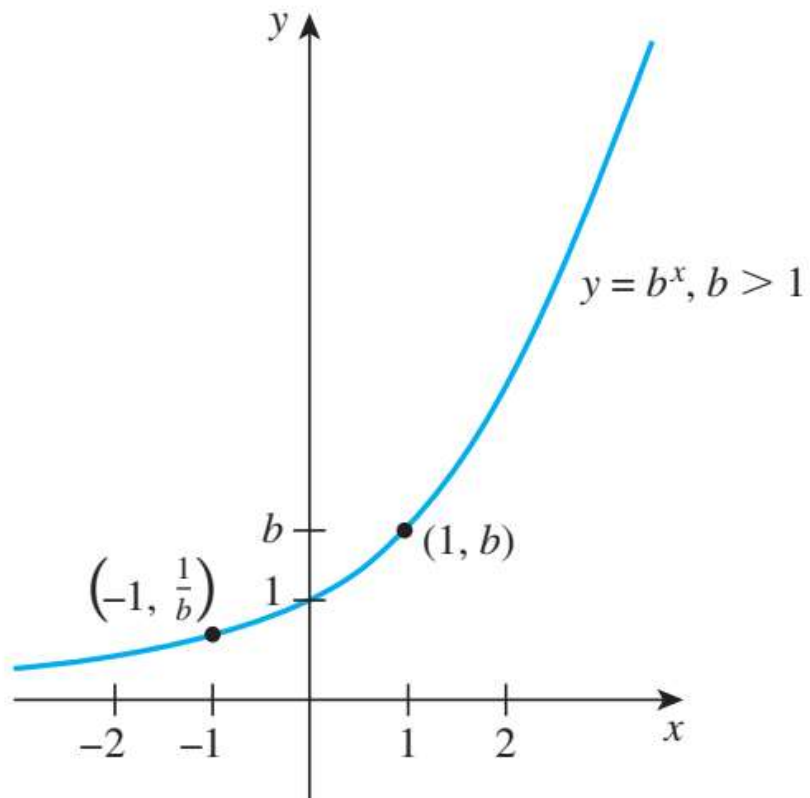
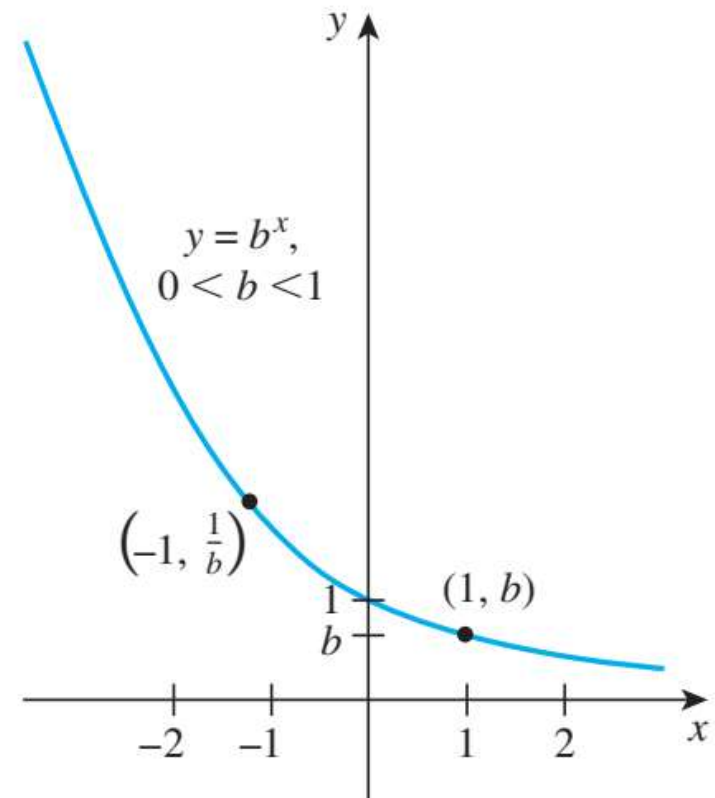


FIGURE 11.4.1 The Exponential Function with Base 2



Graph of the exponential function
with base $b > 1$

(a)



Graph of the exponential function
with base b where $0 < b < 1$

(b)

FIGURE 11.4.2 Graphs of Exponential Functions

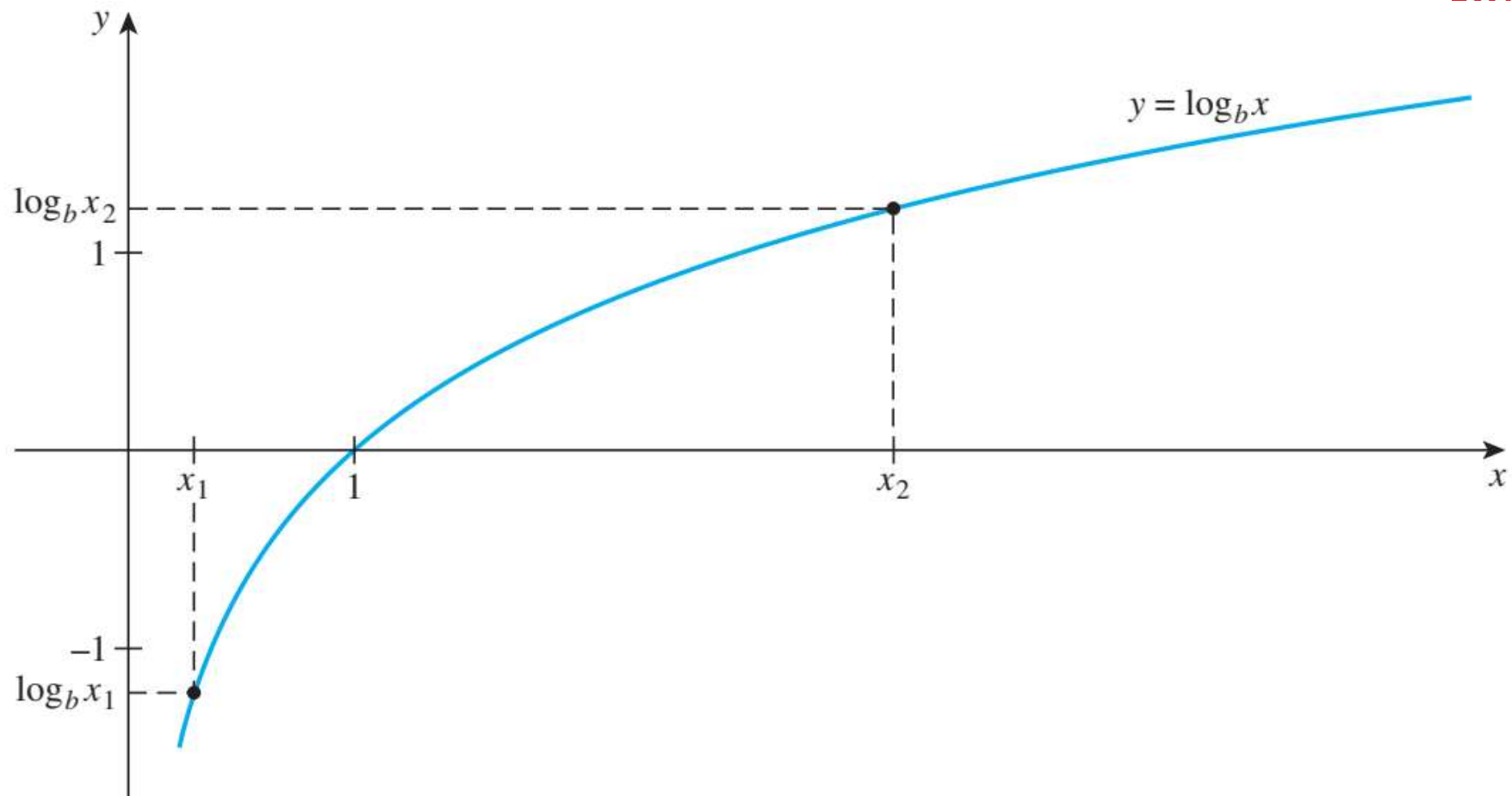


FIGURE 11.4.3 The Graph of the Logarithmic Function with Base $b > 1$

Example 11.4.1

Base 2 Logarithms of Numbers between Two Consecutive Powers of 2

Prove the following property:

a.

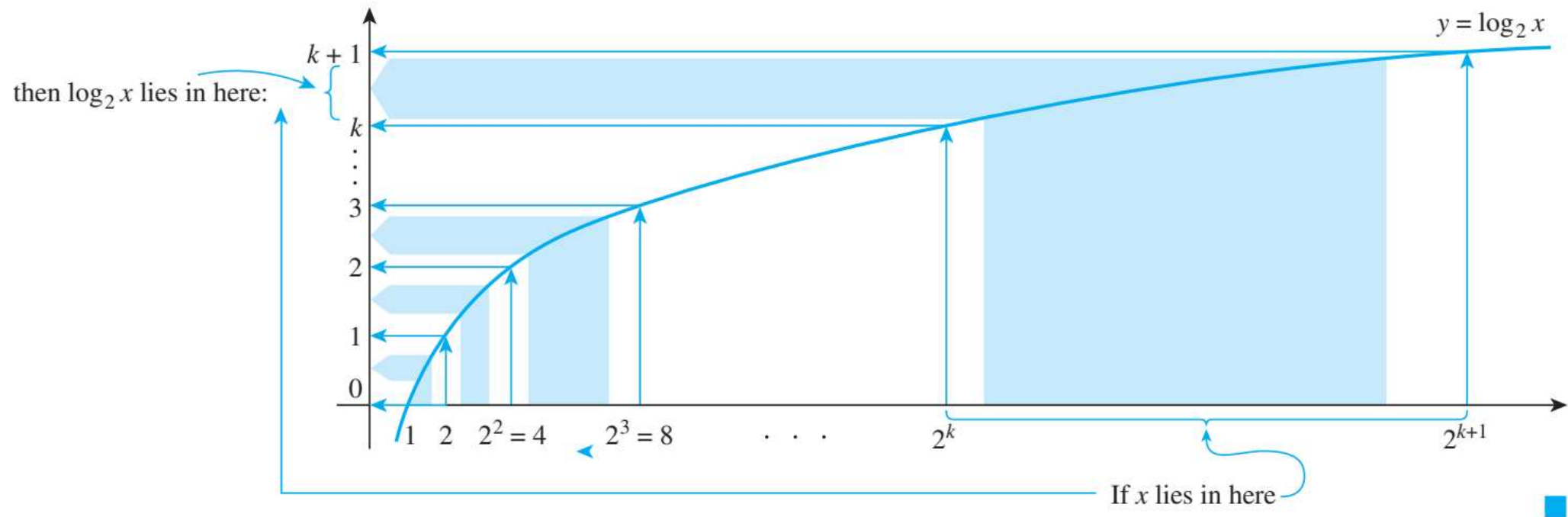
If k is an integer and x is a real number with

$$2^k \leq x < 2^{k+1}, \text{ then } \lfloor \log_2 x \rfloor = k.$$

11.4.2

b. Describe property (11.4.2) in words and give a graphical interpretation of the property for $x > 1$.

A graphical interpretation follows:



Example 11.4.2

When $\lfloor \log_2(n-1) \rfloor = \lfloor \log_2 n \rfloor$

Prove the following property:

For any odd integer $n > 1$, $\lfloor \log_2(n-1) \rfloor = \lfloor \log_2 n \rfloor$.

11.4.3

Solution If n is an odd integer that is greater than 1, then n lies strictly between two successive powers of 2:

$$2^k < n < 2^{k+1} \quad \text{for some integer } k > 0. \quad 11.4.4$$

It follows that $2^k \leq n - 1$ because $2^k < n$ and both 2^k and n are integers. Consequently,

$$2^k \leq n - 1 < 2^{k+1}. \quad 11.4.5$$

Applying property (11.4.2) to both inequalities (11.4.4) and (11.4.5) gives

$$\lfloor \log_2 n \rfloor = k \quad \text{and also} \quad \lfloor \log_2(n - 1) \rfloor = k.$$

Hence $\lfloor \log_2 n \rfloor = \lfloor \log_2(n - 1) \rfloor$. ■

Application: Number of Bits Needed to Represent an Integer in Binary Notation

Then, by property (11.4.2),

$$k = \lfloor \log_2 n \rfloor,$$

and so the number of binary digits needed to represent n is $\lfloor \log_2 n \rfloor + 1$.

Example 11.4.3

Number of Bits in a Binary Representation

How many binary digits are needed to represent 52,837 in binary notation?

Solution If you compute the logarithm with base 2 using the formula in part (a) of Theorem 7.2.1 and a calculator or computer that gives you approximate values of logarithms with base 10, you find that

$$\log_2(52,837) \cong \frac{\log_{10}(52,837)}{\log_{10}(2)} \cong \frac{4.722938151}{0.3010299957} \cong 15.7.$$

Thus the binary representation of 52,837 has $\lfloor 15.7 \rfloor + 1 = 15 + 1 = 16$ binary digits. ■

Example 11.4.4

A Recurrence Relation with a Logarithmic Solution

Define a sequence $a_1, a_2, a_3, \dots$ recursively as follows:

$$a_1 = 1$$

$$a_k = 2a_{\lfloor k/2 \rfloor} \quad \text{for each integer } k \geq 2.$$

- a. Use iteration to guess an explicit formula for this sequence.
- b. Use strong mathematical induction to confirm the correctness of the formula obtained in part (a).



Exponential and Logarithmic Orders

For all real numbers b and r with $b > 1$ and $r > 0$, there is a positive real number s such that

$$\log_b x \leq x^r \quad \text{for every real number } x \geq s. \quad 11.4.9$$

and
$$x^r \leq b^x \quad \text{for every real number } x \geq s. \quad 11.4.10$$

These statements have the following implications for O -notation.

For all real numbers b and r with $b > 1$ and $r > 0$,

$$\log_b n \text{ is } O(n^r) \quad 11.4.11$$

and
$$n^r \text{ is } O(b^n) \quad 11.4.12$$

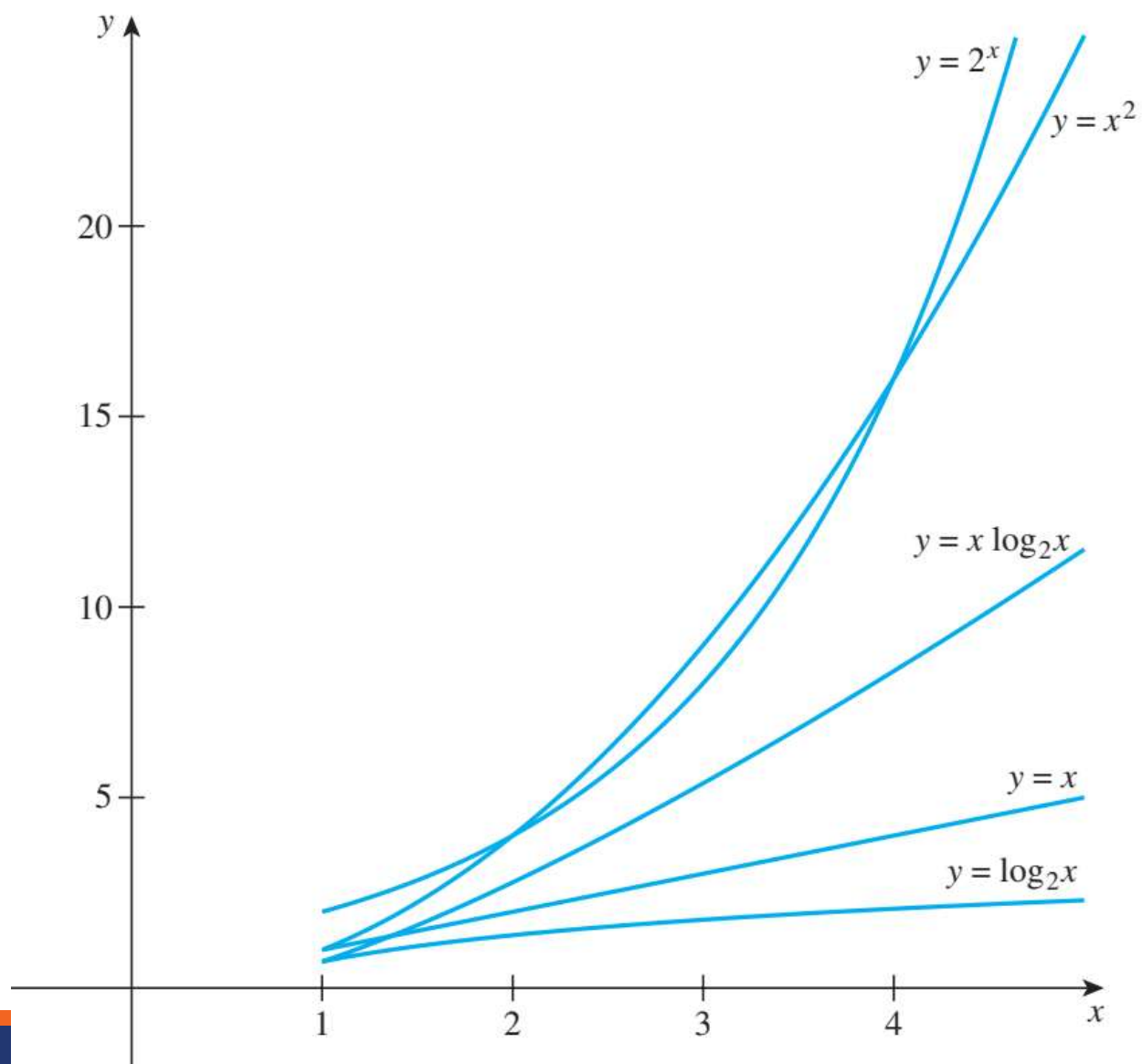
For every real number b with $b > 1$, there is a positive real number s such that for every real number $x \geq s$,

$$x \leq x \log_b x \leq x^2. \quad 11.4.13$$

The O -notation versions of these facts are as follows:

For every real number $b > 1$,

$$n \text{ is } O(n \log_b n) \text{ and } n \log_b n \text{ is } O(n^2). \quad 11.4.14$$



Example 11.4.5

Orders That Involve Logarithmic Functions

- a. Show that if b and c are any real numbers with $b > 1$ and $c > 1$, then $\log_b n$ and $\log_c n$ have the same order.
- b. Show that $n + n \log_2 n$ is $\Theta(n \log_2 n)$.

Solution

a. By part (d) of Theorem 7.2.1,

$$\log_b n = \frac{\log_c n}{\log_c b} = \left(\frac{1}{\log_c b} \right) \log_c n.$$

Since both $b > 1$ and $c > 1$, then $\log_c b > 0$, and so $\frac{1}{\log_c b} > 0$. Thus, by Theorem 11.2.8(c),

$$\left(\frac{1}{\log_c b} \right) \log_c n \text{ is } \Theta(\log_c n), \text{ and so } \log_b n \text{ is } \Theta(\log_c n).$$

Exactly the same reasoning with b and c interchanged gives that

$$\log_c n \text{ is } \Theta(\log_b n).$$

Thus $\log_b n$ and $\log_c n$ have the same order.

- b. By Theorem 11.2.7, n is $\Theta(n)$ and $n \log_2 n$ is $\Theta(n \log_2 n)$. In addition, by property 11.4.13, there is a positive real number r such that for every integer $n \geq r$, $n \leq n \log_2 n$. Thus it follows from Theorem 11.2.9(c) that $n + n \log_2 n$ is $\Theta(n \log_2 n)$. ■

Example 11.4.6

An Exponential Order

Show that $n!$ is $O(n^n)$.

Solution

For any positive integer n ,

$$\begin{aligned} n! &= \underbrace{n \cdot (n-1) \cdot (n-2) \cdot 2 \cdot 1}_{n \text{ factors}} \\ &\leq \underbrace{n \cdot n \cdot n \cdot n \cdot n}_{n \text{ factors}} = n^n \end{aligned}$$

because $(n-1) \leq n$, $(n-2) \leq n$, $\dots$, $2 \leq n$, and $1 \leq n$. Let $B = 1$ and $b = 1$. Then, since $n! \geq 0$ when $n \geq b$,

$$0 \leq n! \leq n^n \quad \text{for every integer } n \geq b.$$

Thus $n!$ is $O(n^n)$ by definition of O -notation. ■

The area underneath the graph of $y = 1/x$ between $x = 1$ and $x = n$ equals $\ln n$, where $\ln n = \log_e n$. This fact is illustrated in Figure 11.4.5.

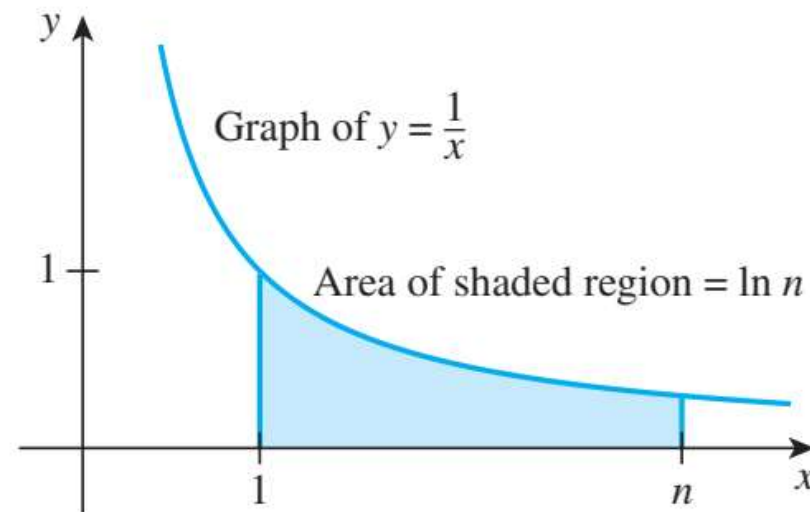


FIGURE 11.4.5 Area under graph of $y = \frac{1}{x}$ between $x = 1$ and $x = n$

Example 11.4.7

Order of a Harmonic Sum

Sums of the form $1 + \frac{1}{2} + \cdots + \frac{1}{n}$ are called *harmonic sums*. They occur in the analysis of various computer algorithms such as quick sort. Show that $1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}$ is $\Omega(\ln n)$ by performing the steps shown below:

- a. Interpret Figure 11.4.6 to show that

$$\frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} \leq \ln n.$$

and

$$\ln n \leq 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}.$$

- b. Show that if n is an integer that is at least 3, then $1 \leq \ln n$.
- c. Deduce from (a) and (b) that if the integer n is greater than or equal to 3, then

$$\ln n \leq 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} \leq 2 \ln n.$$

- d. Deduce from (c) that

$$1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} \text{ is } \Theta(\ln n).$$

Solution

- a. Figure 11.4.6(a) shows rectangles whose bases are the intervals between each pair of integers from 1 to n and whose heights are the heights of the graph of $y = 1/x$ above the right-hand endpoints of the intervals. Figure 11.4.6(b) shows rectangles with the same bases but whose heights are the heights of the graph above the left-hand endpoints of the intervals.

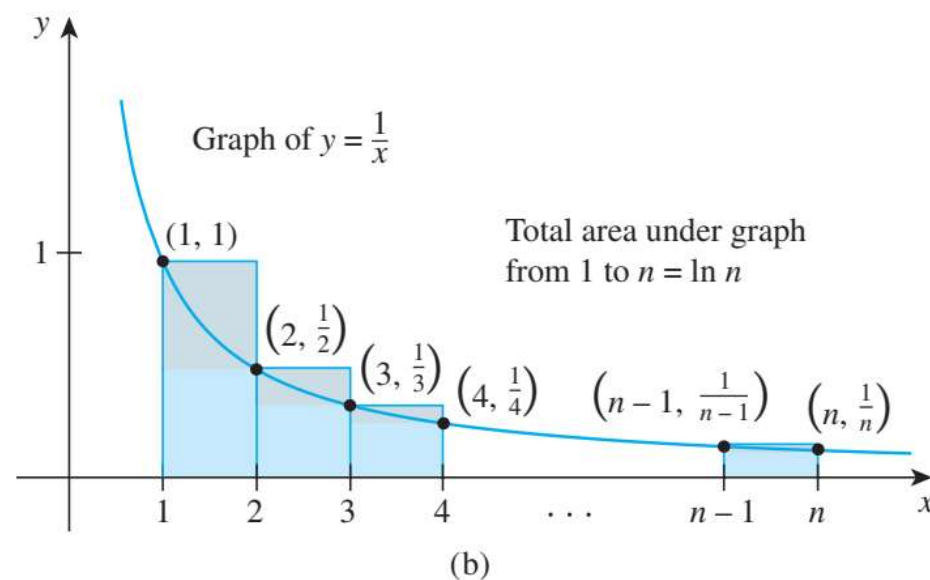
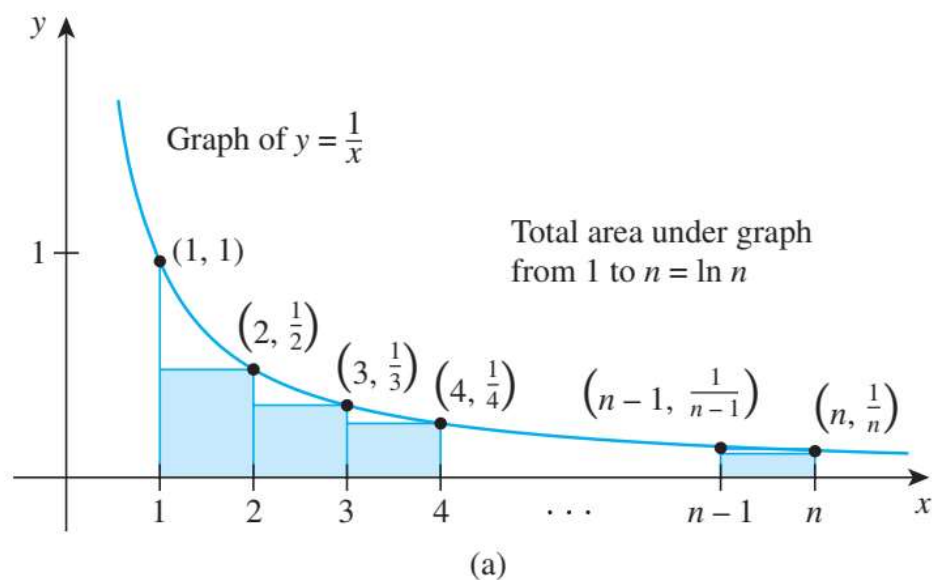


FIGURE 11.4.6

the area of the rectangle from 1 to 2 is $\frac{1}{2}$;

the area of the rectangle from 2 to 3 is $\frac{1}{3}$;

$\vdots$

the area of the rectangle from $n - 1$ to n is $\frac{1}{n}$.

So the sum of the areas of all the rectangles is $\frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}$. From the picture it is clear that this sum is less than the area underneath the graph of f between $x = 1$ and $x = n$, which is known to equal $\ln n$. Hence

$$\frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} \leq \ln n.$$

A similar analysis of the areas of the combined blue and gray rectangles in Figure 11.4.6(b) shows that

$$\ln n \leq 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}.$$

- b. Suppose n is an integer and $n \geq 3$. Since $e \cong 2.718$, then $n \geq e$. Now the logarithmic function with base e is strictly increasing. Thus since $e \leq n$, then $1 = \ln e \leq \ln n$.
- c. By part (a),

$$\frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} \leq \ln n,$$

and by part (b),

$$1 \leq \ln n.$$

Adding these two inequalities together gives

$$1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} \leq 2 \ln n \quad \text{for each integer } n \geq 3.$$

- d. Combining the results of parts (a) and (c) leads to the conclusion that for every integer $n \geq 3$,

$$\ln n \leq 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} \leq 2 \ln n.$$

Let $A = 1$, $B = 2$, and $k = 3$. Then $\ln n > 0$ for $n \geq 3$ and

$$A \ln n \leq 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} \leq B \ln n \quad \text{for every integer } n > k.$$

Hence by definition of Θ -notation,

$$1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} \text{ is } \Theta(\ln n).$$



TEST YOURSELF

1. The domain of any exponential function is _____, and its range is _____.
2. The domain of any logarithmic function is _____, and its range is _____.
3. If k is an integer and $2^k \leq x < 2^{k+1}$, then $\lfloor \log_2 x \rfloor =$ _____.
4. If b is a real number with $b > 1$, then there is a positive real number s with the property that for any real number x that is greater than s , when the quantities x , x^2 , $\log x$, and $x \log x$ are arranged in order of increasing size, the result is _____.
5. If n is a positive integer, then $1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}$ has order _____.

EXERCISE SET 11.4

Graph each function defined in 1–8.

1. $f(x) = 3^x$ for each real number x
2. $g(x) = \left(\frac{1}{3}\right)^x$ for each real number x
3. $h(x) = \log_{10} x$ for each positive real number x
4. $k(x) = \log_2 x$ for each positive real number x
5. $F(x) = \lfloor \log_2 x \rfloor$ for each positive real number x
6. $G(x) = \lceil \log_2 x \rceil$ for each positive real number x
7. $H(x) = x \log_2 x$ for each positive real number x
8. $K(x) = x \log_{10} x$ for each positive real number x

9. The scale of the graph shown in Figure 11.4.1 is one-fourth inch to each unit. If the point $(2, 2^{64})$ is plotted on the graph of $y = 2^x$, how many miles will it lie above the horizontal axis? What is the ratio of the height of the point to the distance of the earth from the sun? (There are 12 inches per foot and 5,280 feet per mile. The earth is approximately 93,000,000 miles from the sun on average.) ($\frac{1}{4}$ inch $\cong$ 0.635 cm, 1 mile $\cong$ 0.62 km)

- 10. a.** Use the definition of logarithm to show that $\log_b b^x = x$ for every real number x .
- b.** Use the definition of logarithm to show that $b^{\log_b x} = x$ for every positive real number x .
- c.** By the result of exercise 28 in Section 7.3, if $f: X \rightarrow Y$ and $g: Y \rightarrow X$ are functions and $g \circ f = I_X$ and $f \circ g = I_Y$, then f and g are inverse functions. Use this result to show that $\log_b$ and $\exp_b$ (the exponential function with base b) are inverse functions.

- H 13.** Suppose a positive real number x satisfies the inequality $10^m \leq x < 10^{m+1}$ where m is an integer. What can be inferred about $\lceil \log_{10} x \rceil$? Justify your answer.
- 14. a.** Prove that if x is a positive real number and k is a nonnegative integer such that $2^{k-1} < x \leq 2^k$, then $\lceil \log_2 x \rceil = k$.
- b.** Describe in words the statement proved in part (a).
- 15.** If n is an odd integer and $n > 1$, is $\lceil \log_2(n-1) \rceil = \lceil \log_2(n) \rceil$? Justify your answer.
- H 16.** If n is an odd integer and $n > 1$, is $\lceil \log_2(n+1) \rceil = \lceil \log_2(n) \rceil$? Justify your answer.
- 17.** If n is an odd integer and $n > 1$, is $\lfloor \log_2(n+1) \rfloor = \lfloor \log_2(n) \rfloor$? Justify your answer.

In each of 21 and 22, a sequence is specified by a recurrence relation and initial conditions. In each case, (a) use iteration to guess an explicit formula for the sequence; (b) use strong mathematical induction to confirm the correctness of the formula you obtained in part (a).

21. $a_k = a_{\lfloor k/2 \rfloor} + 2$, for each integer $k \geq 2$
 $a_1 = 1$

22. $b_k = b_{\lfloor k/2 \rfloor} + 1$, for each integer $k \geq 2$
 $b_1 = 1$.

H 23. Define a sequence $c_1, c_2, c_3, \dots$ recursively as follows:

$$c_1 = 0$$

$$c_k = c_{\lfloor k/2 \rfloor} + k, \quad \text{for each integer } k \geq 2.$$

Use strong mathematical induction to show that $c_n \leq n^2$ for every integer $n \geq 1$.

Use Theorems 11.2.7–11.2.9 and properties 11.4.11, 11.4.12, and 11.4.13 to derive each statement in 27–30.

27. $2n + \log_2 n$ is $\Theta(n)$

28. $n^2 + 5n \log_2 n$ is $\Theta(n^2)$

29. $n^2 + 2^n$ is $\Theta(2^n)$

H 30. 2^{n+1} is $\Theta(2^n)$

H 31. Show that 4^n is not $O(2^n)$.

Prove each of the statements in 32–37, assuming n is an integer variable that takes positive integer values. Use identities from Section 5.2 as needed.

32. $1 + 2 + 2^2 + 2^3 + \cdots + 2^n$ is $\Theta(2^n)$.

H 33. $4 + 4^2 + 4^3 + \cdots + 4^n$ is $\Theta(4^n)$.

34. $2 + 2 \cdot 3^2 + 2 \cdot 3^4 + \cdots + 2 \cdot 3^{2n}$ is $\Theta(3^{2n})$.

35. $\frac{1}{5} + \frac{4}{5^2} + \frac{4^2}{5^3} + \cdots + \frac{4^n}{5^{n+1}}$ is $\Theta(1)$.

36. $n + \frac{n}{2} + \frac{n}{4} + \cdots + \frac{n}{2^n}$ is $\Theta(n)$.

37. $\frac{2n}{3} + \frac{2n}{3^2} + \frac{2n}{3^3} + \cdots + \frac{2n}{3^n}$ is $\Theta(n)$.

38. Quantities of the form

$$k_1 n + k_2 n \log n \quad \text{for positive integers } k_1, k_2, \text{ and } n$$

arise in the analysis of the merge sort algorithm in computer science. Show that for any positive integer k ,

$$k_1 n + k_2 n \log_2 n \text{ is } \Theta(n \log_2 n).$$

39. Calculate the values of the harmonic sums

$$1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} \quad \text{for } n = 2, 3, 4, \text{ and } 5.$$

40. Use part (d) of Example 11.4.7 to show that

$$n + \frac{n}{2} + \frac{n}{3} + \cdots + \frac{n}{n} \text{ is } \Theta(n \ln n).$$

41. Show that $\lfloor \log_2 n \rfloor$ is $\Theta(\log_2 n)$.

42. Show that $\lceil \log_2 n \rceil$ is $\Theta(\log_2 n)$.

43. Prove by mathematical induction that $n \leq 10^n$ for every integer $n \geq 1$.

- H 44.** Prove by mathematical induction that $\log_2 n \leq n$ for every integer $n \geq 1$.
- H 45.** Show that if n is a variable that takes positive integer values, then 2^n is $O(n!)$.
- 46.** Let n be a variable that takes positive integer values.
- a.** Use Example 11.4.6 to show that $\log_2(n!)$ is $O(n \log_2 n)$.
 - H b.** Show that $n^n \leq (n!)^2$ for every integer $n \geq 1$.
 - c.** Use part (b) to show that $\log_2(n!)$ is $\Omega(n \log_2 n)$.
 - d.** Use parts (a) and (c) to find an order for $\log_2(n!)$.



Application: Analysis of Algorithm Efficiency II

Binary Search

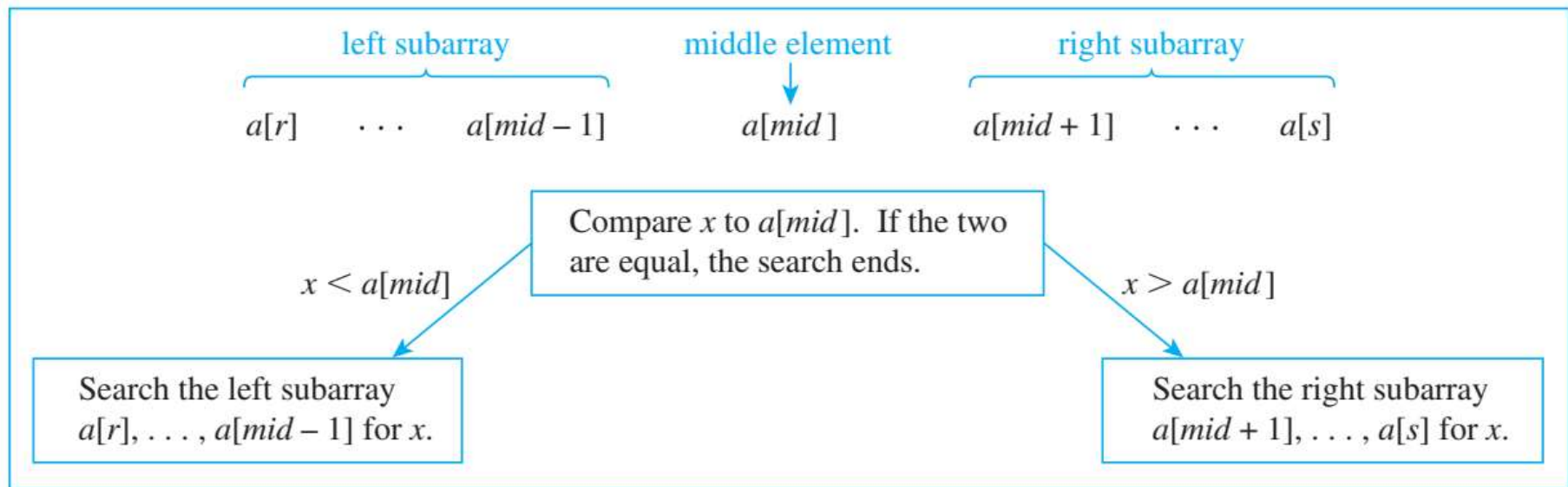


FIGURE 11.5.1 One Iteration of the Binary Search Process

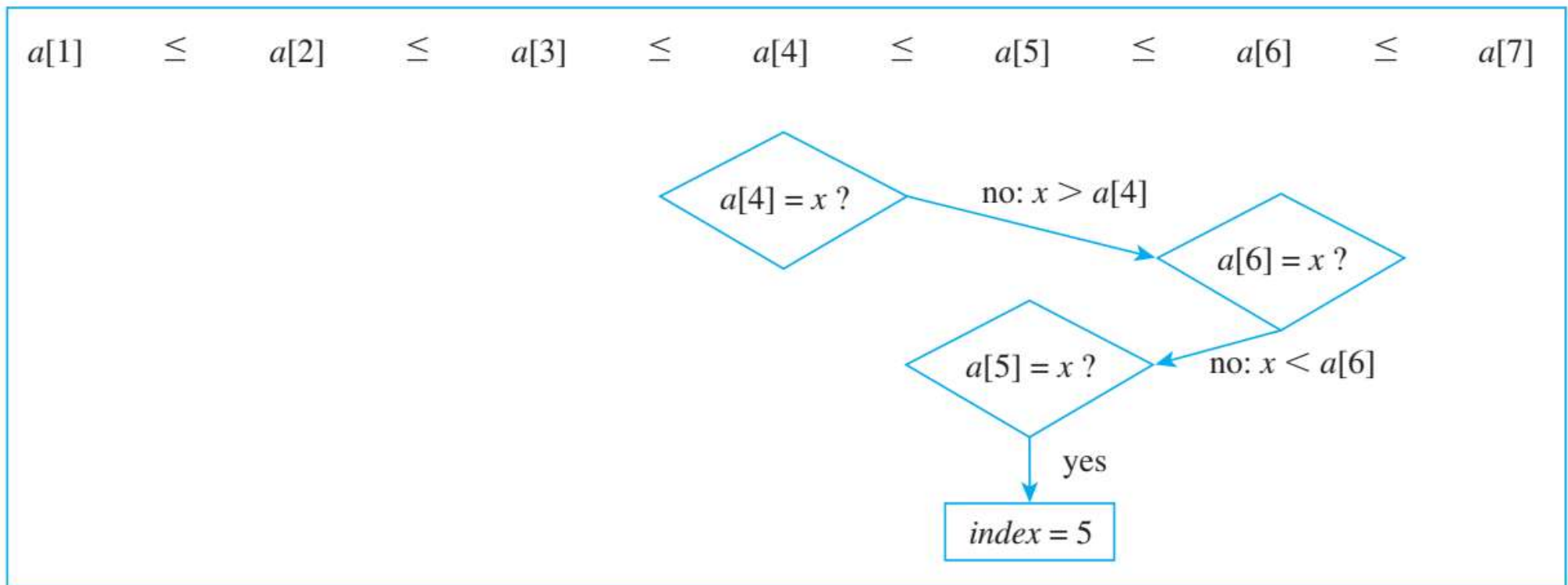


FIGURE 11.5.2 Binary Search of $a[1], a[2], \dots, a[7]$ for x (for $x = a[5]$)

Algorithm 11.5.1 Binary Search

[The aim of this algorithm is to search for an element x in an ascending array of elements $a[1], a[2], \dots, a[n]$. If x is found, the variable $index$ is set equal to the index of the array element where x is located. If x is not found, $index$ is not changed from its initial value, which is 0. The variables bot and top denote the bottom and top indexes of the array currently being examined.]

Input: n [a positive integer], $a[1], a[2], \dots, a[n]$ [an array of data items given in ascending order], x [a data item of the same data type as the elements of the array]

(continued on page 816)

Algorithm Body:

$index := 0, bot := 1, top := n$

[Compute the middle index of the array, mid. Compare x to $a[mid]$. If the two are equal, the search is successful. If not, repeat the process either for the lower or for the upper subarray, either giving top the new value $mid - 1$ or giving bot the new value $mid + 1$. Each iteration of the loop either decreases the value of top or increases the value of bot . Thus, if the looping is not stopped by success in the search process, the value of top eventually becomes less than the value of bot , which stops the looping process and shows that x is not an element of the array.]

while ($top \geq bot$ and $index = 0$)

$mid := \left\lfloor \frac{bot + top}{2} \right\rfloor$

if $a[mid] = x$ **then** $index := mid$

if $a[mid] > x$

then $top := mid - 1$

else $bot := mid + 1$

end while

*[If $index$ has the value 0 at this point, then the **while** loop was not entered because $top < bot$, and so x is not in the array. Otherwise, $index$ gives the index of the array where x is located.]*

Output: $index$ [a nonnegative integer]

Example 11.5.1

Tracing the Binary Search Algorithm

Trace the action of Algorithm 11.5.1 on the variables *index*, *bot*, *top*, *mid*, and the values of *x* given in (a) and (b) below for the input array

$a[1] = \text{Ann}$, $a[2] = \text{Dawn}$, $a[3] = \text{Erik}$, $a[4] = \text{Gail}$, $a[5] = \text{Juan}$,
 $a[6] = \text{Matt}$, $a[7] = \text{Max}$, $a[8] = \text{Rita}$, $a[9] = \text{Tsuji}$, $a[10] = \text{Yuen}$,

where alphabetical ordering is used to compare elements of the array.

- a. $x = \text{Erik}$ b. $x = \text{Sara}$

Solution

a.

<i>index</i>	0			3
<i>bot</i>	1		3	
<i>top</i>	10	4		
<i>mid</i>		5	2	3
<i>x</i>	Erik			

b.

<i>index</i>	0			
<i>bot</i>	1	6	9	
<i>top</i>	10			8
<i>mid</i>		5	8	9
<i>x</i>	Sara			

TEST YOURSELF

1. To solve a problem using a divide-and-conquer algorithm, you reduce it to a fixed number of smaller problems of the same kind, which can themselves be _____, and so forth until _____.
2. To search an array using the binary search algorithm in each step, you compare a middle element of the array to _____. If the middle element is less than _____, you _____, and if the middle element is greater than _____, you _____.
3. The worst-case order of the binary search algorithm is _____.
4. To sort an array using the merge sort algorithm, in each step until the last one you split the array into approximately two equal sections and sort each section using _____. Then you _____ the two sorted sections.
5. The worst-case order of the merge sort algorithm is _____.

EXERCISE SET 11.5

1. Use the facts that $\log_2 10 \cong 3.32$ and that for each real number a , $\log_2(10^a) = a \log_2 10$ to find $\log_2(1,000)$, $\log_2(1,000,000)$, and $\log_2(1,000,000,000,000)$.
2. Suppose an algorithm requires $c\lceil\log_2 n\rceil$ operations when performed with an input of size n (where c is a constant).
 - a. By what factor will the number of operations increase when the input size is increased from m to m^2 (where m is a positive integer power of 2)?
 - b. By what factor will the number of operations increase when the input size is increased from m to m^{10} (where m is a positive integer power of 2)?
 - c. When n increases from 128 ($= 2^7$) to 268,435,456 ($= 2^{28}$), by what factor is $c\lceil\log_2 n\rceil$ increased?

3. For what values of n is an algorithm that requires n operations more efficient than an algorithm that requires $\lfloor 50 \log_2 n \rfloor$ operations?
4. For what values of n is an algorithm that requires $\lfloor n^2/10 \rfloor$ operations more efficient than an algorithm that requires $\lfloor n \log_2 n \rfloor$ operations?

In 5 and 6, trace the action of the binary search algorithm (Algorithm 11.5.1) on the variables *index*, *bot*, *top*, *mid*, and the given values of x for the input array $a[1] = \text{Chia}$, $a[2] = \text{Doug}$, $a[3] = \text{Jan}$, $a[4] = \text{Jim}$, $a[5] = \text{Jose}$, $a[6] = \text{Mary}$, $a[7] = \text{Rob}$, $a[8] = \text{Roy}$, $a[9] = \text{Sue}$, $a[10] = \text{Usha}$, where alphabetical ordering is used to compare elements of the array.

5. a. $x = \text{Chia}$ b. $x = \text{Max}$
6. a. $x = \text{Amanda}$ b. $x = \text{Roy}$
7. Suppose *bot* and *top* are positive integers with $\text{bot} \leq \text{top}$. Consider the array

$$a[\text{bot}], a[\text{bot} + 1], \dots, a[\text{top}].$$

- a. How many elements are in this array?
- b. Show that if the number of elements in the array is odd, then the quantity $\text{bot} + \text{top}$ is even.

- c. Show that if the number of elements in the array is even, then the quantity $bot + top$ is odd.

Exercises 8–11 refer to the following algorithm segment. For each positive integer n , let a_n be the number of iterations of the **while** loop.

```
while ( $n > 0$ )
     $n := n \text{ div } 2$ 
end while
```

8. Trace the action of this algorithm segment on n when the initial value of n is 27.
9. Find a recurrence relation for a_n .
10. Find an explicit formula for a_n .
11. Find an order for this algorithm segment.

Exercises 12–15 refer to the following algorithm segment.
For each positive integer n , let b_n be the number of iterations of the **while** loop.

```

while ( $n > 0$ )
     $n := n \text{ div } 3$ 
end while
    
```

12. Trace the action of this algorithm segment on n when the initial value of n is 424.
13. Find a recurrence relation for b_n .
- H 14.**
 - a. Use iteration to guess an explicit formula for b_n .
 - b. Prove that if k is an integer and x is a real number with $3^k \leq x < 3^{k+1}$, then $\lfloor \log_3 x \rfloor = k$.
 - c. Prove that for every integer $m \geq 1$,

$$\lfloor \log_3(3m) \rfloor = \lfloor \log_3(3m+1) \rfloor = \lfloor \log_3(3m+2) \rfloor.$$
 - d. Prove the correctness of the formula you found in part (a).
15. Find an order for the algorithm segment.
16. Complete the proof of case 2 of the strong induction argument in Example 11.5.5. In other words, show that if k is an odd integer and $w_i = \lfloor \log_2 i \rfloor + 1$ for every integer i with $1 \leq i \leq k$, then $w_{k+1} = \lfloor \log_2 k + 1 \rfloor + 1$.

Exercises 3 and 4 illustrate that for relatively small values of n , algorithms with larger orders can be more efficient than algorithms with smaller orders. Use a graphing calculator or computer to answer these questions.



Questions?